Distributed Recovery of RIPv2 (RFC 2082) Authentication Keys

A Master–Worker Brute-Force Cluster over TCP

Santiago Hernández A01787550

TC2007B — Network Security • September 22, 2026

1 Introduction

This report documents a distributed brute-force attack that recovers the authentication key protecting RIPv2 routing updates captured in a controlled lab. RIPv2 supports a keyed message-digest authentication mode (RFC 2082 Baker and Atkinson (1997)); the captures use six-character keys. We first reverse-engineer the exact digest construction from a packet capture (Section 2), then build a master–worker cluster that partitions the keyspace across TCP-connected workers (Section 3, Section 4), tolerates node failure (Section 5), and we characterize its throughput (Section 6).

▶ **TODO:** write the motivation / threat-model framing here

2 RIP Packet and MD5 Authentication: Reverse-Engineering and Validation

Packet layout. Each captured update is a RIPv2 *Response* (command 2) sent to the multicast group 224.0.0.9 on UDP/520. Authentication of type 3 (Keyed Message Digest) places an authentication *header* as the first route-table entry and appends an authentication *trailer* carrying the digest. The observed size-06 packet decomposes as in Table 1.

Region	Bytes	Contents (size-06 capture)
RIP header	4	command=2, version=2, routing-domain=0
Auth header (RTE)	20	AFI=0xFFFF, auth-type=3, pkt-len=44, key-id=1, auth-len=20, seq, reserved
Route entry	20	the advertised route
Auth trailer	20	AFI=0xFFFF, type=0x0001, then the 16-byte digest

Table 1: RIPv2/RFC 2082 authenticated packet layout (64-byte payload).

Digest construction (the key finding). The assignment brief names "HMAC-MD5", but the captured trailers are *not* reproducible with HMAC. RFC 2082 instead specifies a *keyed* MD5: the 16-byte digest field is first overwritten with the key, and a plain MD5 is taken over the whole packet. Concretely, writing D for the payload bytes from the RIP header through the trailer's `AFI/type` field (i.e. everything except the 16 digest bytes) and K_{16} for the ASCII key right-padded with zero bytes to 16:

$$\text{digest} \;=\; \text{MD5}\big(\, D \,\|\, K_{16} \,\big). \tag{2.1}$$

We tested both constructions and both candidate alphabets by brute force; only (2.1) over a **base-36** alphabet (lowercase letters and digits, *not* hexadecimal) reproduces the captured digest.

Recovered key and validation. The single-threaded oracle recovers `ykusrq` (index 1,618,816,210 of 36^6). Validation is not a single-packet coincidence: the same key reproduces the digest of **all 21** authenticated packets in the capture, each with a distinct sequence number and payload. Table 2 summarizes the corrected parameters.

Parameter	Brief says	Actual (measured)	Evidence
Construction	HMAC-MD5	keyed-MD5 (RFC 2082)	only (2.1) matches
Alphabet	hex / "letters+numbers"	base-36	hex space exhausted, no match
Key length	6	6	filename `size_06` + match
Key (06_01)	—	`ykusrq`	21/21 packets validate

Table 2: Reverse-engineered authentication parameters.

▶ **TODO:** discuss the security implications of the construction and short keys

3 TCP Coordination Protocol and Message Types

The master and workers communicate over raw TCP sockets using a line-oriented protocol: each message is one JSON object terminated by a newline, which makes framing trivial and the traffic human-readable during debugging. The master never blocks a fast worker: chunks are handed out *on request*, so faster nodes simply request more often. Table 3 lists the message types.

Message	Direction	Purpose / fields
`HELLO`	W→M	join; worker id, host, pid, cores
`PARAMS`	M→W	target data, digest, alphabet, length, construction
`REQUEST`	W→M	ask for a keyspace chunk
`ASSIGN`	M→W	{chunk id, start, end}
`PROGRESS`	W→M	{chunk, tried, rate}; doubles as a heartbeat
`HEARTBEAT`	W→M	liveness while idle between chunks
`DONE`	W→M	chunk exhausted with no match
`FOUND`	W→M	{chunk, key, index} on a match
`NOWORK`	M→W	queue momentarily empty; retry
`STOP`	M→W	global halt (key found, or keyspace exhausted)

Table 3: Master–worker message protocol.

Lifecycle. A worker sends `HELLO`, receives `PARAMS`, then loops: `REQUEST` → `ASSIGN`, crack the chunk in sub-batches while emitting `PROGRESS`, and report `DONE` or `FOUND`. On `FOUND` the master *re-verifies* the claimed key against the captured digest before trusting it, then broadcasts `STOP` to every connected worker so no node keeps hashing. The master node is never idle: it launches local worker subprocesses that connect back over loopback and crack alongside the remote workers.

▶ **TODO:** optional: justify the protocol/encoding design choice in your words

4 Keyspace Partition Strategy

The search space is every length-6 string over the base-36 alphabet, of size $N = 36^6 = 2{,}176{,}782{,}336$. Each integer index $i \in [0, N)$ maps to a key by little-endian positional encoding over the alphabet, so a *contiguous* index interval corresponds to a set of distinct keys with no overlap between intervals. The master divides $[0, N)$ into fixed-size chunks of C keys, giving $\lceil N/C \rceil$ chunks (e.g. $C = 5 \times 10^6$ yields 436 chunks), and serves chunk ids from a queue.

Algorithm 1 Dynamic, non-overlapping assignment

```
pending ← queue(0, 1, ..., ⌈N/C⌉ − 1);  inflight ← ∅
while a REQUEST arrives and not found do
  if pending ≠ ∅ then
    c ← pop(pending);  inflight[c] ← (worker, now)
    send ASSIGN(c, cC, min((c+1)C, N))
  else if inflight ≠ ∅ then
    send NOWORK {outstanding chunk may be requeued}
  else
    mark exhausted;  broadcast STOP
  end if
end while
```

Dynamic (pull-based) assignment rather than a static up-front split means heterogeneous or contended workers stay balanced automatically, and it composes cleanly with failure recovery: a chunk is only ever "done" when its `DONE` arrives, so a lost chunk can be safely re-queued (Section 5).

▶ **TODO:** optional: analyze the chunk-size trade-off

5 Node-Death Detection and Reassignment

No assigned chunk may be silently lost, or the search could terminate reporting "not found" while skipping the key. The master defends against two failure modes, both of which requeue the dead node's chunk to the *front* of `pending` so a surviving worker picks it up:

1. **Hard crash / kill.** The TCP socket closes; the master's read returns EOF, and the disconnect handler requeues every chunk that worker held.
2. **Freeze / network partition.** The socket stays open but `PROGRESS` heartbeats stop. A monitor thread requeues any chunk whose last heartbeat is older than a timeout T (default 5 s) and marks the node dead.

Empirical check. In a controlled run with three workers, freezing one with `SIGSTOP` and killing another with `SIGKILL` produced the expected master events:

```
TIMEOUT: worker freeze-w1 silent >4.0s, requeued chunk 19
worker kill-w2 disconnected; requeued chunks [40]
```

Both chunks were subsequently completed by the surviving worker, so coverage of the keyspace remained total.

▶ **TODO:** optional: discuss timeout tuning trade-offs

6 Benchmark Metrics

All measurements target the size-06 digest and the keyed-MD5 construction on a host with 10 logical cores. Rates are in millions of keys hashed per second (Mkeys/s).

Measurement	Aggregate	Per node/core
BM1 single-core baseline	2.55	2.55
BM2 scaling, 8 processes	10.67	1.33
BM2 scaling, 10 processes	10.95	1.09
BM3 distributed, 8 TCP workers	12.56	≈1.75

Table 4: Hashing throughput (Mkeys/s). Full sweep in `results/benchmarks.txt`.

CPU-bound vs. network-bound. Instrumenting each worker to split its wall time into hashing versus time blocked on the socket shows the workload is overwhelmingly compute-bound: across the distributed run, aggregate compute time was 69.1 s against 0.03 s of socket wait, i.e. **99.96%** of time is spent hashing. The coordination protocol is therefore not the bottleneck; total throughput scales with available CPU cores until the host saturates (Table 4, where per-core rate falls once the process count exceeds the physical core budget).

End-to-end. The full recovery of `ykusrq` took 738 s single-threaded versus 142 s on the contended cluster; an uncontended ∼12.6 Mkeys/s cluster covers the whole 2.18×10^9 space in well under that.

▶ **TODO:** optional: generate the scaling figure from notebooks/graphing.ipynb and interpret the saturation point

7 Conclusion and Deferred Work

We recovered a RIPv2/RFC 2082 authentication key by reverse-engineering the keyed-MD5 construction (correcting the brief on both the MAC and the alphabet) and validating it against every authenticated packet in the capture. A pull-based master–worker cluster over TCP, with heartbeat- and disconnect-based failure recovery, reproduces the same key and is strongly CPU-bound.

Deferred: live route injection. The final offensive step — forging an authenticated RIPv2 update with the recovered key and injecting it into a live FRR/Mininet topology with Scapy — is implemented as a documented stub (`src/inject_route.py`) and was intentionally *not* executed here, as it requires the Mininet sandbox. See the `TODO` in that file.

▶ **TODO:** write closing remarks and mitigation recommendations

References

Baker, F. and Atkinson, R. (1997). RIP-2 MD5 Authentication. Technical Report RFC 2082, Internet Engineering Task Force. `https://www.rfc-editor.org/rfc/rfc2082`.